EMBEDDED SYSTEMS & INTERNET OF THINGS (IoT)

TASK 1 PROJECT REPORT

Prepared by: Junior Embedded Engineer

Topic: Introduction to Embedded Systems & Smart Sensor System Design

Target Hardware: Arduino UNO (Simulated on Tinkercad Circuits)

1. Introduction Section (Research)

What is an Embedded System?

An embedded system is a specialized computer system composed of hardware and software optimized to perform a dedicated function within a larger electrical or mechanical architecture. Unlike general-purpose computers, embedded systems are engineered to operate with high reliability, minimal latency, and low power consumption under real-time constraints.

Real-World Applications

- **Smart Home:** Automated HVAC climate control, smart lighting systems, and security authentication devices.
- **Automotive:** Engine Control Units (ECUs), Anti-lock Braking Systems (ABS), and airbag control modules.
- **Healthcare Devices:** Wearable heart-rate monitors, digital glucose meters, and automated insulin delivery pumps.
- **Robotics & Industrial:** Multi-axis robotic arms, automated guided vehicles (AGVs), and programmable logic controllers (PLCs).

Microcontroller vs. Microprocessor Comparison

Parameter	Microcontroller (MCU)	Microprocessor (MPU)
Internal Architecture	CPU, RAM, ROM, and I/O peripherals integrated on a single IC chip.	Consists solely of the CPU; relies on external RAM, ROM, and bus controllers.
Power & Cost	Low power consumption, low cost, optimized for dedicated control tasks.	Higher power consumption, higher computational throughput, higher cost.
Typical Target	Embedded controllers (e.g., ATmega328P, ESP32).	Personal computers and heavy OS applications (e.g., Intel Core, ARM Cortex-A).

2. Component Study

Microcontrollers

- **Arduino UNO (ATmega328P):** An 8-bit AVR board operating at 16 MHz with 32 KB flash memory. Excellent for entry-level prototyping and basic I/O interfacing.
- **ESP32:** A 32-bit dual-core board running at 240 MHz with built-in Wi-Fi and Bluetooth, targeted for connected IoT products.
- **Raspberry Pi Pico (RP2040):** A 32-bit dual-core ARM Cortex-M0+ microcontroller designed for high-performance low-power computing.

Basic Sensors

- **Temperature Sensor (TMP36):** An analog precision sensor outputting a voltage directly proportional to ambient temperature in Celsius.
- **Motion Sensor (PIR):** A passive infrared sensor that detects movement by measuring changes in IR radiation levels.
- **Light Sensor (LDR):** A photoresistor whose electrical resistance decreases as incident light intensity increases.

3. Mini Project – Smart Temperature Monitor

System Working Principle

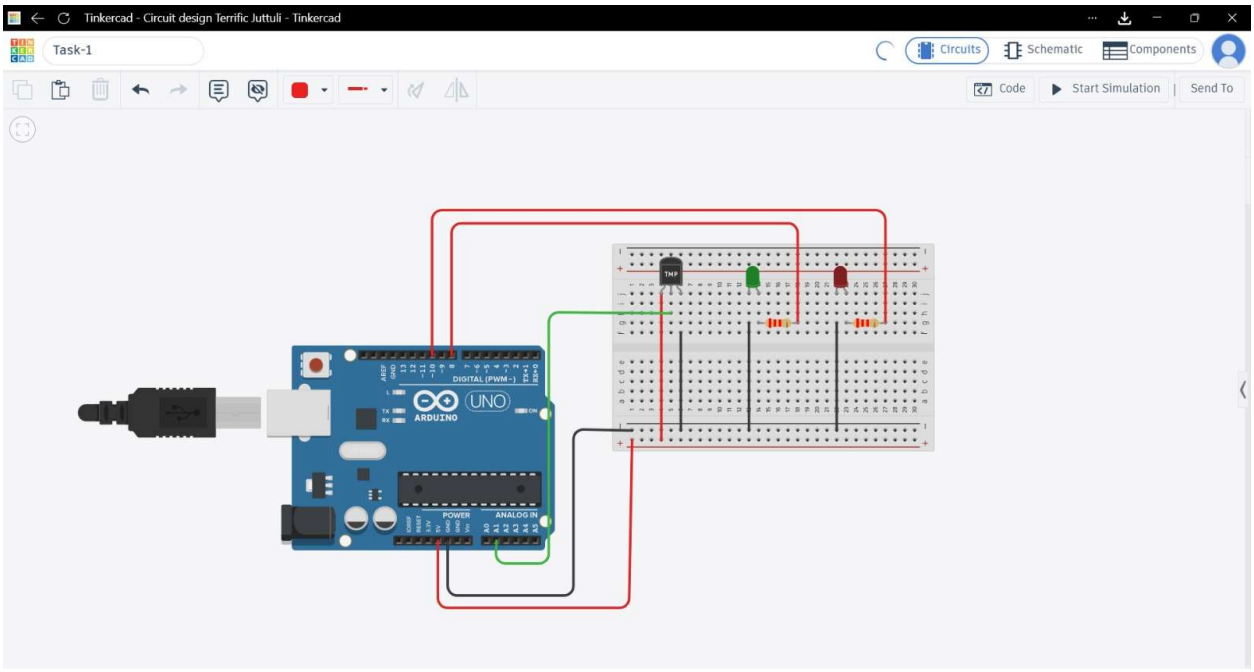
The Smart Temperature Monitor acquires environmental temperature data using a TMP36 sensor connected to an analog input pin. The Arduino converts the raw 10-bit Analog-to-Digital Converter (ADC) value into an equivalent voltage and computes the temperature in degrees Celsius (˚C). If the temperature reaches or exceeds the threshold of $30.0°C$, a Red Alert LED illuminates. Otherwise, a Green LED remains lit to indicate normal operating conditions.

Circuit Pin Mapping

Component	**Pin / Terminal**	**Destination**
Power Rail	Red Rail (+) / Blue Rail (-)	Arduino **5V** / Arduino **GND**
TMP36 Sensor	Pin 1 (Power / Left)	Red Rail (+5V)
	Pin 2 (Signal / Center)	Arduino Analog Pin **A1**
	Pin 3 (GND / Right)	Blue Rail (GND)
Red LED (Alert)	Anode (+ via 2.2 kΩ resistor)	Arduino Digital Pin **10**
	Cathode (-)	Blue Rail (GND)

Component	Pin / Terminal	Destination
Green LED (Safe)	Anode (+ via 2.2 kΩ resistor)	Arduino Digital Pin **8**
	Cathode (-)	Blue Rail (GND)

Tinkercad Circuit diagram :



Source Code:

```
/*
 * Project Title: Smart Temperature Monitor
 * Target Board: Arduino UNO
 * Sensor: TMP36 (Connected to A1)
```

```
 * Indicators: Red LED (Pin 10), Green LED (Pin 8)
 */

const int tempPin = A1;         // Analog pin reading sensor signal
const int redLed = 10;          // Pin driving Alert LED
const int greenLed = 8;         // Pin driving Normal LED
const float tempThreshold = 30.0;// Threshold trigger in Celsius

void setup() {
  pinMode(redLed, OUTPUT);
  pinMode(greenLed, OUTPUT);
  Serial.begin(9600);           // Baud rate for telemetry logging
}

void loop() {
  // Read raw 10-bit ADC output (0 - 1023)
  int sensorVal = analogRead(tempPin);

  // Convert raw reading to input voltage
  float voltage = (sensorVal * 5.0) / 1024.0;

  // Convert voltage to Celsius using TMP36 scale factor (10 mV/°C with 500 mV offset)
  float temperatureC = (voltage - 0.5) * 100.0;

  // Output sensor telemetry to Serial Monitor
  Serial.print("Current Temp: ");
```

```
  Serial.print(temperatureC);
  Serial.println(" °C");
  // Threshold decision logic
  if (temperatureC >= tempThreshold) {
    digitalWrite(redLed, HIGH);   // Activate alert state
    digitalWrite(greenLed, LOW);
  } else {
    digitalWrite(redLed, LOW);
    digitalWrite(greenLed, HIGH);  // Activate normal state
  }
  delay(1000);                  // 1-second sampling delay
}
```

Formula Derivations:

- **ADC to Voltage:**

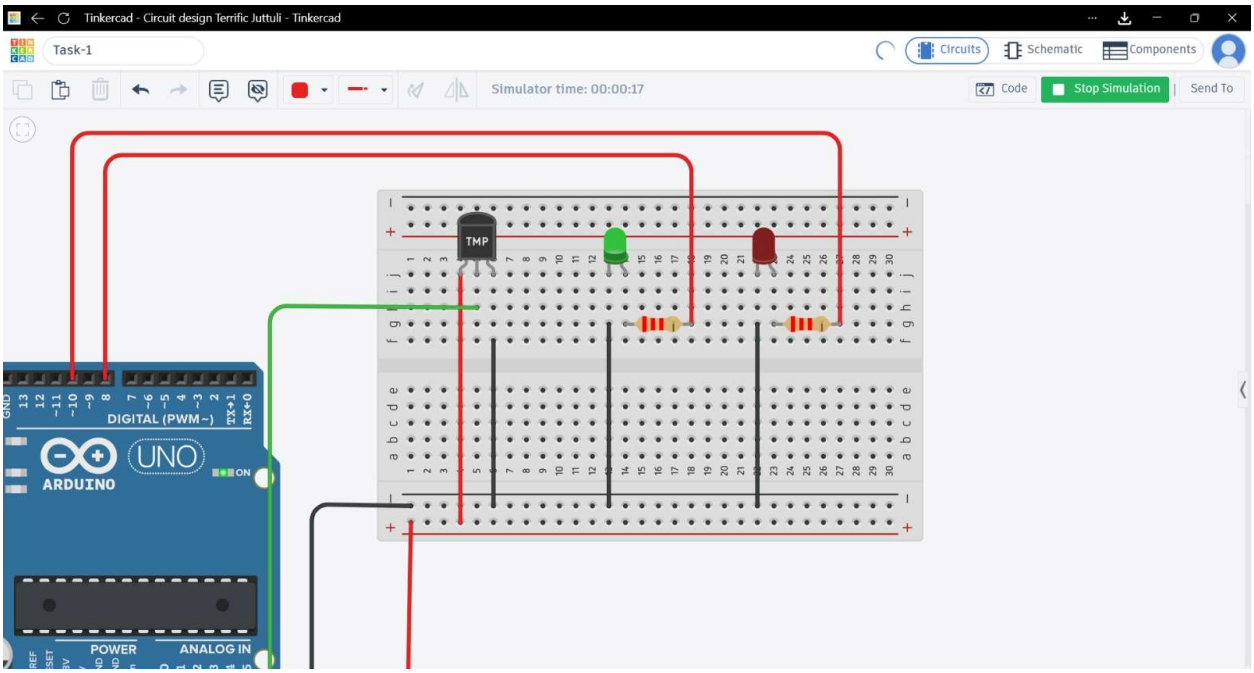
$$V_{out} = \frac{\text{ADC Reading} \times 5.0\text{V}}{1024.0}$$

- **Voltage to Temperature:**

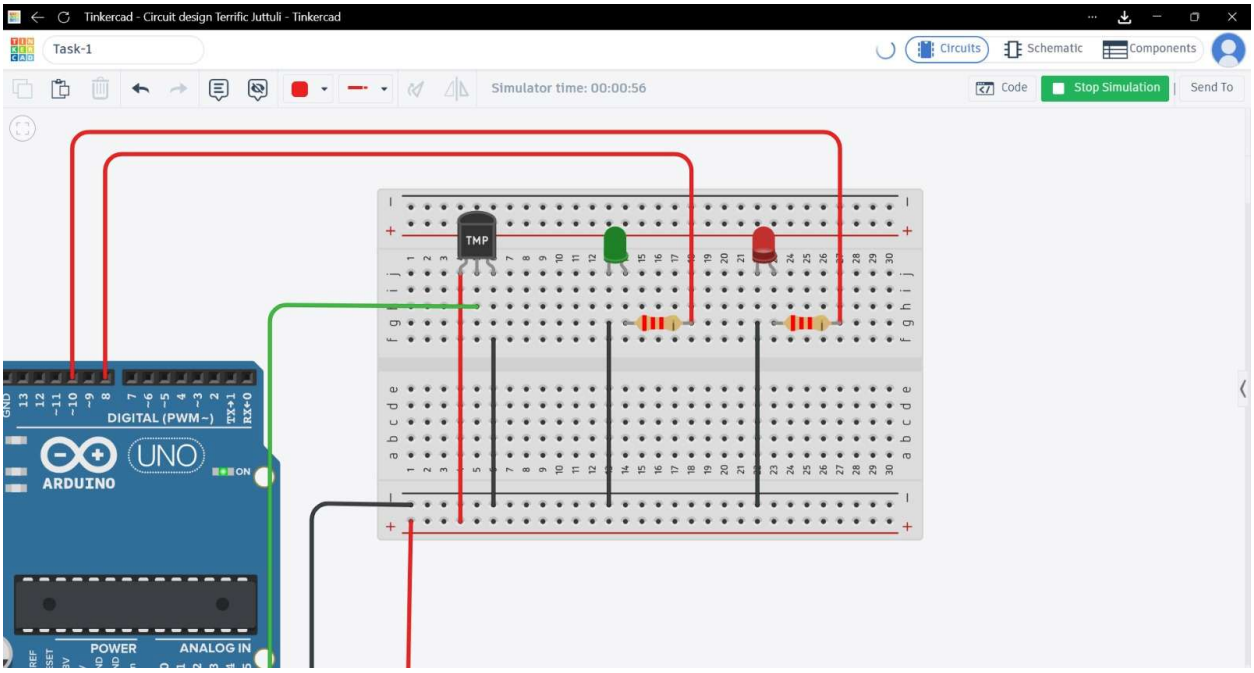
$$T(\circ \text{C}) = (V_{out} - 0.5) \times 100$$

OUTPUT :

For Temperature below 28.0 C :



For above 28.0 C :



output link :

https://www.tinkercad.com/things/cav6gECGUzS-task-1/editel?returnTo=%2Fdashboard%2Fdesigns%2Fcircuits&sharecode=Wpa5bY-kx4I1yOnriqfNuelHYDZPef8YsalYgYFEnzo

4. Potential Real-World Applications

- **Industrial Equipment Safety:** Automatic shutdown mechanisms triggered when high-power motors exceed safe operational temperatures.
- **Smart Data Centers:** Monitoring rack thermal profiles to dynamically engage backup cooling units.
- **Pharmaceutical Logistics:** Cold-chain monitoring to preserve temperature-sensitive vaccines and reagents during transport.